arXiv:2608.16319v2 [cs.LG] 7 Sep 2026

Advancing Open and Reproducible Relational Learning: RelArena-α, TabPFN-Rel and RPI

Core Contributors: **Adrian Hayler**[1] **Klemens Flöge**[1] **Alan Arazi**[1]
External Collaborators: **Rishabh Ranjan**[2] **Jure Leskovec**[2,3]
Research & Project Leads: **Lennart Purucker**[1,4] **Frank Hutter**[1,4,5] **Noah Hollmann**[1]
and the Prior Labs Team[1]

[1]Prior Labs [2]Stanford University [3]NVIDIA
[4]University of Freiburg [5]ELLIS Institute Tübingen







Abstract

This first release of Prior Labs in relational learning shows our continued commitment to open science. We open-source three pieces of software that we expect to accelerate research in the field towards meaningful real-world impact. We aim to steer further development based on feedback from, and in collaboration with, the community. Given the early stage of development, our α-release targets researchers and early-adopting practitioners.

RelArena-α. Over the past years, a variety of datasets and tasks for relational learning have emerged [1, 2, 3, 4], but the community has not converged on a reliable, reproducible way to compare different methods on these tasks. Our α-release, RelArena-α, provides a unified framework for running and comparing baselines on RelBench v1 [1] by standardizing data loading, evaluation protocols, tuning regimes, and support for systems with custom tuning, inspired by established tabular benchmarks such as TabArena [5]. We plan to work with the research community to further develop RelArena-α into a catalyst for progress in the relational learning community.

TabPFN-Rel. We release the initial version of TabPFN-Rel, a purpose-built relational harness for TabPFN-3. Currently ranked first among models on RelArena-α, TabPFN-Rel makes key improvements upon RDBLearn [6]. Beyond its ranking, TabPFN-Rel serves as a strong baseline, adding to the growing evidence that flattening a relational database into a single table remains competitive with specialized relational architectures on real-world tasks [6, 7, 8, 9].

RPI. To facilitate adoption of relational learning methods in research and industry, we release an initial α-version of our **R**elational **P**redictive **I**nterface, RPI, an open-source, model-agnostic interface that enables early adopters to easily define problems on new databases and apply any model implemented in RelArena-α, including TabPFN-Rel, to these problems.

https://github.com/PriorLabs/relarena

1 The Current State of Relational Learning

Relational learning has attracted increased interest and research output in recent years. Nevertheless, we believe that progress in the field has been slowed and real-world impact hindered by the following interlocking factors.

- **Methods are not reproducible.** Several of the strongest-reported methods cannot be independently reproduced, as training scripts are unavailable and tuning procedures are undisclosed.
- **Methods are not comparable.** Reported results are often obtained under differing evaluation protocols and tuning budgets, making end-to-end performance comparisons inherently confounded. As a result, apparent differences in *model* performance may reflect differences in the surrounding *system* rather than differences between the models themselves (e.g., tuning, validation protocol). The community lacks standardized guidance for comparing models and systems under controlled evaluation regimes that allow reliable comparisons.
- **No shared aggregation standard.** The community lacks a common convention for aggregating results across tasks, making overall performance claims difficult to compare and interpret.
- **Lack of pragmatic baselines.** Feature-engineering baselines prevalent in industry are underrepresented in academic evaluations and are frequently implemented without sufficient care.
- **No path from paper to practice.** Published methods lack accessible interfaces that would allow practitioners to apply them to their own databases.

Table 1 summarizes, for the methods discussed throughout this section, which of these issues apply.

Table 1: **Reproducibility and comparability of methods reporting RelBench v1 results.** State at the time of submission. We summarize code availability, tuning practices, evaluation regimes, aggregation methods, task coverage, and packaging, which we discuss in detail in Section 1. Search-space sizes are taken from documented procedures or inferred from released per-task configurations where the tuning procedure is undisclosed. RelArena-α uses method-specific trial budgets chosen to approximately balance tuning compute (see Appendix B).

Method	Code for all reported tasks	Per-task tuning	Tuning documented	Search space size	RelBench eval. regime	Regression aggregate	#Tasks reported	Installable package
GraphSAGE	✓	✗	–	–	✓	mean MAE	21	✗
RelGNN	✗	✓	✗	up to 25k	✓	–	21	✗
RelGT	✓	✓	✓	9	✓	–	21	✗
KumoRFM	✗	✓	✗	up to 10	✗	MAE / RDL	21	✗
KumoRFM-2	✗	✓	✗	up to 384	✗	MAE / LGBM	21	✗
RT	✓	✗	–	–	✗	mean R^2	18	✗
PluRel	✓	✗	–	–	✗	mean R^2	18	✗
RT-J	✓	✓	✓	32	✗	σ-norm. MAE	21	✗
RDBLearn	✓	✓	✓	9	✓	MAE / AG	16	✓
RelArena-α	✓	✓	✓	$\sim$matched	✓	Elo (+others)	21	✓

1.1 Methods are not reproducible

Impossible to reproduce. While many relational learning methods release all code required for reproduction, the level of reproducibility varies across methods. In some cases, released implementations provide dataset-specific model checkpoints but not the corresponding training scripts [10], despite community interest in obtaining the training code [11]. In other cases, foundation models are made available through an API, but the provided scripts do not cover all tasks reported in the paper [12]. These differences in availability make it difficult to verify results under a common experimental setup and to perform controlled comparisons across methods.

Opaque tuning regimes. Similarly, even when results can be reproduced via an API or model checkpoint, performance is often contingent on dataset-specific hyperparameters, which are often selected from grids spanning hundreds or thousands of configurations. Large configuration spaces are not problematic in themselves, but without a documented tuning regime it is unclear how extensively they were explored, how final configurations were selected and if the tuning regime is transferable to real-world settings. For KumoRFM-2, the released per-task configurations vary along five

dimensions whose observed values alone induce a grid of 384 combinations [13]; the released RelGNN checkpoints vary along nine dimensions whose observed values induce a grid of over 25,000 combinations [14]. Neither the publications nor the underlying codebases explain how these hyperparameters were obtained [10, 12], and public requests for clarification remained unresolved at the time of this release [14, 15]. Without this information, it is difficult to assess how much of the reported performance reflects the method itself rather than method-agnostic optimizations.

1.2 Methods are not comparable

Unfortunately, it has become common practice in the relational learning community to copy self-reported baseline results for method comparisons [10, 16, 17, 12, 6, 7, 8, 18, 9], a practice we also followed in pursuit of "comparability" in a recent publication [19]. This practice may partly stem from the reasons outlined above, which sometimes force authors to copy reported baseline results if they want to, or have to, compare against a specific method. Still, even assuming baseline results are reproducible, the reported baselines are, with few exceptions, not comparable, as we outline below.

Differences in evaluation regimes. Defining a task in relational learning is complex and not standardized. As a result, subtle differences in evaluation regimes have emerged over time across method releases. These subtle differences can easily lead to significant downstream performance differences, undermining the scientific integrity of the method comparison.
To illustrate, consider the entity-level forecasting tasks in RelBench [1], which have been widely used by the community. The entity-level forecasting tasks in RelBench freeze the database at a fixed, pre-defined test cut-off, independent of the specific timestamp of a test entity. In particular, test entities do not have access to the label of other test entities with earlier timestamps. This evaluation regime is defined in RelBench and enforced by the provided data-loading logic. However, not every method directly uses RelBench's data loading, which enables methods to utilize the additional information ingested into the database between the fixed test cut-off and the entity's timestamp. While most methods follow the former evaluation regime, we observed that KumoRFM-2 [12] (and likely its predecessor, KumoRFM [17]) as well as methods derived from the relational transformer architecture [20, 21, 18] follow the latter regime. While this difference materially affects only one of the seven RelBench v1 databases, it still has a very meaningful impact on aggregate statistics. On every database other than `rel-f1`, all test entities share a single timestamp equal to the test cut-off, so both regimes expose the same database state. On the `rel-f1` tasks, however, test timestamps span three to six years, and methods following the latter regime gain access to years of additional signal: evaluating the same model under RelBench's regime instead reduces its results by over ten ROC AUC points on the two classification tasks and increases its MAE by over 40% on the regression task [19, 12]. Both evaluation regimes are valid, but the difference in additional context makes them incomparable. In a fair comparison, all methods would use the same evaluation regime.

Differences in data states. The RelBench maintainers actively address issues in databases and tasks as they are discovered. Defects of this kind are to be expected in any benchmark of this scale. However, as a community, we need to account for these fixes in method comparisons; they change the data state and may invalidate older baseline results, which, in turn, need to be recomputed.
To illustrate, consider the temporal-leakage fix released in early 2026 [22], which regenerated the labels for the `rel-event/user-ignore` task. When affected methods are re-evaluated on the corrected task, their ROC AUC scores are consistently two to six points lower than the originally reported values [1, 10, 16] (cf. our re-runs in Appendix D). Methods evaluated against different versions of the RelBench package may therefore be evaluated on materially different data states, rendering direct comparisons of their self-reported results impossible.

Differences in tuning. Across many subfields of machine learning, model performance is known to be sensitive to dataset-specific hyperparameters [23, 24, 25, 26, 5]. Comparing a tuned to an untuned method confounds the comparison. The tuning alone could explain the performance difference. The same holds when comparing a method tuned under one regime with one tuned under a different regime (e.g., random search vs. Bayesian optimization). Hence, it is vital to standardize the tuning regime across models to eliminate tuning as a confounding factor. At the same time, it is vital not to suppress research ingenuity and to enable benchmarking of novel tuning regimes. Unfortunately, it remains acceptable in the relational learning literature for authors to demonstrate the merit of their isolated contribution with comparisons confounded by the tuning regimes.

Even popular methods, such as RelGNN [10] and RelGT [16], follow this pattern. Both build upon and compare against the relational deep learning (RDL) baseline [27] by modifying the underlying model architecture. Additionally, both methods apply tuning to improve model performance. Yet the authors compare only against results reported in Robinson et al. [1], which were obtained without tuning. "RDL" could have been run with the same generic search space as RelGNN or RelGT, as they use the same underlying data pipeline; or RelGNN and RelGT could have positioned their tuning as an integral part of their contributions. Without fair, rigorous method comparisons, it is impossible to determine whether improvements over baselines come from a novel tuning regime or methodological improvements; this misleads researchers and confuses practitioners.

1.3 No shared aggregation standard

Even with a fixed evaluation regime and a standardized tuning budget, per-dataset results still have to be aggregated before general conclusions can be drawn, and here the community has no shared convention. Work that uses RelBench reports the same per-task metrics (ROC AUC for classification, MAE for regression), but summarizes them in incompatible ways. While classification results are aggregated consistently (as the unweighted mean of raw ROC AUC scores [1, 12, 20]), for regression, the aggregates use MAE normalized by the performance of different reference models before averaging: LightGBM [12, 8], the RDL baseline [17], a non-relational AutoGluon [6], or RelGT [7]. Other approaches include means of raw MAE across tasks whose scales differ by orders of magnitude [1], a geometric mean of raw MAE [9], standardization by the target's standard deviation [18], and replacing the metric with R^2 altogether [20, 21]. Further summaries include average relative improvement over a baseline [1, 17], win counts [10, 16], and mean ranks [12, 6, 8, 9]. Several papers also compute their aggregates over subsets of the available tasks [20, 21, 6, 7, 9], dropping between three and six of the 21 tasks for reasons ranging from suspected data leakage (see also Section 1.2) to no stated rationale.

These summaries are not interchangeable and carry different interpretations. Averaging raw ROC AUC values gives higher weight to tasks with larger absolute differences, while down-weighting tasks with meaningful differences that are small in absolute value. A reference-model-normalized MAE has a different interpretation because it weights absolute differences relative to the performance of the reference model (e.g., across two datasets, it rewards the same absolute difference less when the reference model performs worse on one). Summaries over an incomplete result table can additionally favor methods that report only subsets of tasks on which they perform well.

Because each paper uses its own summary, aggregate numbers are rarely comparable across papers, leaving the field without a common quantity against which method development can be measured. This is less of a problem in adjacent communities, which have converged on shared approaches to summarizing per-dataset results: language model comparisons use Elo ratings derived from pairwise outcomes [28], while the tabular literature also reports normalized scores and mean ranks, along with significance tests across datasets [5, 29]. A comparable convention for relational learning would make general trends easier to interpret and give the community a common target to optimize for.

1.4 Lack of pragmatic baselines

Researchers must compare against pragmatic baselines that industry practitioners currently use to solve prediction tasks on relational databases (RDBs). Such methods typically convert the relational learning task into a tabular one by flattening the RDB into a table [27, 1, 3]. On such a table, traditional tabular learning methods can be applied to produce predictions. There are many ways to convert an RDB into a flat table; these generally fall into automated and manual feature engineering.

Manual feature engineering. An important contribution of RelBench v1 [1] is the inclusion of a "data scientist" baseline that combines manual feature engineering with LightGBM [30]. This baseline is particularly valuable because it reflects how industry practitioners would approach these tasks. At the same time, it should be viewed as a standardized, reproducible reference rather than an estimate of the best performance achievable through manual feature engineering. In practice, data scientists typically refine their feature engineering based on validation set performance. RelBench's division of the data scientist workflow into fixed sequential steps cannot capture this iterative feedback loop [1]. Recent work suggests that iterative feature refinement can yield substantial performance improvements [8].

Automated feature engineering. While automated feature engineering approaches [31, 32, 33] have been popular in industry, they have been ignored as possible baselines for years. Only recently have they been evaluated as baselines in the relational learning literature, where they have achieved competitive performance from the outset [6, 7]. Before this, baselines that lift tabular learners to relational learning via automated feature engineering provided the model with only a handful of raw columns from the entity's own table [1, 4], making for a weak and unrepresentative baseline. While these baselines significantly underestimate the power of tabular learning for relational learning tasks, they are prominently used to demonstrate the advantages of GNN-based approaches over flattening with tabular learning [1, 4].

In extreme cases, tabular baseline implementations contain obvious errors. For the autocomplete tasks of RelBench v2, we noticed that the reported LightGBM baseline has a 50% ROC-AUC and negative R^2. After a quick investigation, we found that the LightGBM baseline receives only `NaN` inputs at test time, resulting in 50% ROC-AUC and negative R^2. This example illustrates that baselines require the same implementation care and validation as newly proposed methods.

1.5 No path from paper to practice

Despite these shortcomings, many exciting developments in relational learning over the past few years could benefit industry practitioners. At the same time, transferring these methods from research benchmarks to real-world prediction problems remains unnecessarily difficult.

Research methods are typically developed around benchmark tasks represented through task tables [27, 1, 2, 4]. RelBench v1 provides tutorials for extending the benchmark to custom databases and prediction tasks. However, doing so still requires users to write task-specific Python code and interact with RelBench's dataset and task abstractions. This suits researchers extending the benchmark, but it does not provide a dedicated declarative interface through which practitioners can specify a prediction problem on an existing relational database.

To the best of our knowledge, the only attempt to establish a dedicated query language for this purpose has been the Predictive Query Language (PQL) [34], whose implementations remain proprietary. Moreover, PQL's published grammar cannot express some RelBench tasks, whose labels require multi-hop aggregations (`rel-avito/user-clicks` and `rel-trial/study-outcome`).

Application of relational learning methods is further hindered by the absence of a unified model interface comparable to scikit-learn [35] for tabular learning or HuggingFace Transformers [36] for language models. Switching between methods therefore requires custom integration logic for each model and task. Compounding this, currently most relational learning methods are not distributed as installable packages, requiring practitioners to work directly with the individual codebases.

2 Towards Open, Reproducible, and Accessible Relational Learning

Our first open-source release in relational learning is a first step towards addressing the issues raised in the previous section: RelArena-α targets reproducibility and comparability, TabPFN-Rel addresses the lack of pragmatic baselines, and RPI makes relational learning research accessible to industry practitioners. While all released artifacts are still in early development, we believe they could already benefit the community. We ask the community for feedback and contributions to accelerate the development of an open, reproducible, and accessible relational learning ecosystem.

2.1 RelArena-α

Inspired by the success of TabArena [5] and its recent extension BeyondArena [37] in enabling fair and reproducible benchmarking for tabular learning, we introduce RelArena-α, with the goal of bringing the same standard to relational learning. RelArena-α redistributes no data, retrieving all databases at runtime through RelBench under their respective licenses (see Appendix G). For its initial α-release, RelArena-α focuses on RelBench v1's entity-level forecasting tasks. We choose this scope for two reasons. First, entity-level forecasting is by far the most widely used task type in the current relational learning literature [16, 10, 20, 18, 21, 17, 12, 6, 38], making it the natural starting point for establishing a common comparison framework. Second, methods developed for one relational task type do not necessarily transfer directly to others, making it difficult to define a unified benchmark across task types at this stage. We therefore view the current task set as a prag-

matic starting point rather than a definitive benchmark; broader questions around task and dataset curation remain open, as discussed in Section 4. We detail our main contributions below.

Reproducibility of methods. We ensure the reproducibility of methods by re-running all methods reported on RelArena-α through a unified model API. We have invested significant effort to align model implementations with our benchmark API, fix bugs, and, if necessary, reconstruct training scripts that were not provided by the authors. We have spent hundreds of GPU hours producing baseline results that the community can trust, while also providing an accessible way for anyone to reproduce our reported results.

Tuning regimes for models and systems. We enable benchmarking and differentiated comparisons of methods that use a standardized tuning regime or a custom regime within one framework. RelArena-α enables developers to investigate isolated methodological improvements, custom and novel tuning regimes, or co-dependent improvements. To support these different types of research, following TabArena, we categorize method submissions as "models" or "systems":

- **Model submissions** follow a standardized tuning regime, allowing us to make claims about isolated methodological effects. In RelArena-α, the standardized tuning regime includes random/grid search and early stopping on the validation set. A model submission only needs to declare a search space; RelArena-α controls configuration sampling, run scheduling, and selection of the final candidate. Search spaces may not be dataset-specific beyond coarse differentiations based on dataset size, and we provide them for all implemented methods, mirroring the authors' choices where possible.
- **System submissions** may use custom tuning regimes, such as Bayesian optimization or conditional search steps. Comparing system submissions with each other allows us to understand which end-to-end pipeline performs best under the same input/output and time constraints. Between system and model submissions, we can compare final predictive performance, but efficiency and methodological improvements are not directly comparable because they may result from differences in the tuning regime.

This split between model and system submissions allows us to accommodate meaningful, novel research with system submissions while maintaining the ability to draw reliable research conclusions from model submissions. Our formalization of model submissions constitutes a first, significant step towards comparable tuning in the relational learning community. Yet, fully aligning tuning regimes across methods and integrating all best practices remains an open research problem, mainly due to large differences in methodology, run times across methods and datasets, and preprocessing. We discuss this in detail in Appendix B. The long-term goal for benchmarking relational learning should be to converge on best practices for tuning that make model development research meaningful for practitioners, while using system submissions to track the best end-to-end pipelines.

Unified evaluation regime and data state. Through RelArena-α's unified model API, we ensure that all methods receive the same data state during training, tuning, and evaluation, preventing any possible drift between evaluation regimes (e.g., as described in Section 1.2).

Strong set of baselines. We chose our initial RelArena-α baselines based on their authors' self-reported performance on RelBench. RelArena-α implements GNN-based methods, such as GraphSAGE [27], RelGT [16] and RelGNN [10]; relational foundation models, such as RT-PluRel [20, 21]; tabular aggregation-based approaches, such as RDBLearn [6] and TabPFN-Rel [19]; and pragmatic baselines, such as the learning-free constant predictors, which output each task's optimal global or per-entity constant (Section 3). Of these, RT-PluRel is a system submission, and the rest are model submissions. RT-PluRel uses the relational transformer model [20, 18] pretrained on PluRel-generated [21] synthetic data and fine-tuned on the given task with a custom, sequential tuning regime (see Appendix F for details).
The current set of baselines does not reflect the complete state of relational learning methods. We discuss this further in Section 4. Yet, to the best of our knowledge, RelArena-α presents the most comprehensive unified evaluation of relational learning baselines to date. Together with the community, we will continue to grow the set of baselines.

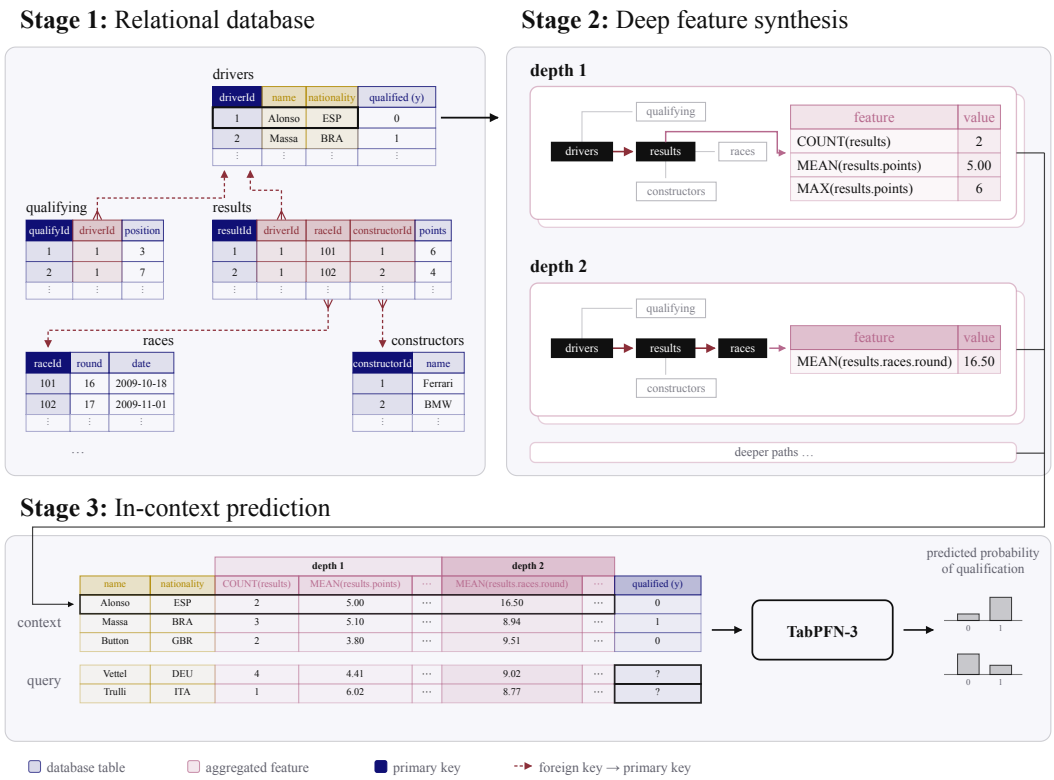


Figure 1: The TabPFN-Rel harness, illustrated on `rel-f1/driver-top3`. Building on RDBLearn [6, 7], TabPFN-Rel uses deep feature synthesis [31] to aggregate relational features along foreign-key paths into a flat table. TabPFN-3 then predicts query labels in-context from labelled context rows. Section 2.2 details our improvements over the RDBLearn pipeline.

Shared evaluation tooling. We integrate RelArena-α directly with TabArena's `bencheval` package [5], the evaluation library behind the TabArena leaderboard. Given a results table produced by RelArena-α, `bencheval` lets us compute a range of aggregate statistics and plots, including average ranks, critical-difference diagrams, leaderboards with bootstrapped Elo ratings (including confidence intervals), pairwise win-rate matrices, and normalized and baseline-relative scores.
As relational benchmarking matures and scales, the importance of trustworthy aggregate statistics will only grow. Likewise, sharing evaluation methodologies across communities is crucial to advancing evaluation research and unifying communication with practitioners.

2.2 TabPFN-Rel

TabPFN-Rel is our relational harness for TabPFN-3, our most recent tabular foundation model (TFM). We visualize the harness in Figure 1. It builds upon RDBLearn [6, 7], which in turn builds upon the popular `featuretools` [31, 39] library. As demonstrated in Section 3, TabPFN-Rel is currently the top-ranked model on RelArena-α, demonstrating again [6, 7] that flattening-based approaches to relational learning can be strong baselines. TabPFN-Rel inherits RDBLearn's core recipe: Each relational prediction task is converted into a flat table by exhaustively aggregating along all join paths implied by the schema's primary–foreign key relationships up to a maximum depth d (deep feature synthesis [31]), with d tuned per task ($d \in \{2, 3, 4\}$). TabPFN-Rel improves upon RDBLearn by:

- **Improved tuning regime:** We resolve data drift issues that previously occurred during tuning by ensuring that the database used during tuning (the inner split) is frozen at the validation cut-off, mirroring how the final evaluation (the outer split) freezes it at the test cut-off, as outlined in Section 1.2. We note that as tuning is automated by RelArena-α, all baseline methods, including RDBLearn, now benefit from this improvement.

- **Improved TFM backbone:** TabPFN-3 is both more capable and scalable than previous TFMs [19] and was co-developed with the relational harness in mind. We make full use of TabPFN-3's improvements by increasing the number of rows fed into the TFM by an order of magnitude and replacing the set of TFM backbones with TabPFN-3. Even with the larger context size, TabPFN-Rel's runtime stays comparable, both due to the removal of one tuning axis (RDBLearn additionally selects among three TFM backbones in its sweep) and a more scalable TFM architecture.
- **Support of text features:** We extend the RDBLearn recipe by re-attaching text columns from the entity table after the `featuretools` preprocessing. We then make use of TabPFN-3-Plus' superior ability to handle text features [19] to further boost performance. We note that this feature is only available via the TabPFN API. For users who are not able to use the API, we also release a text-free version of TabPFN-Rel, which users can run without querying the API.
- **Better context selection:** Lastly, we introduce a novel context selection regime, which replaces the random subsampling used in RDBLearn. By trading off recency and diversity among estimators' contexts, we are able to improve the harness's performance at no additional runtime cost. Additionally, after determining the optimal featurization depth d on the validation set, we also re-use validation examples as additional context for making predictions on the test set. Due to the temporal nature of the forecasting tasks, the more recent validation examples are often particularly informative for making predictions.

2.3 Relational Predictive Interface: RPI

Complementing RelArena-α and TabPFN-Rel, we release an initial version of our Relational Predictive Interface, RPI, to enable the application of relational learning methods, including TabPFN-Rel, to real-world tasks. RPI is fully open-source, model-agnostic, and directly integrated into RelArena-α, allowing users to run any RelArena-α baseline, including hyperparameter tuning, on their own relational prediction problem in two lines of code. It generalizes the process used to generate the entity-level forecasting tasks of RelBench v1 [1], but replaces custom task-generation code with a declarative interface: the database and the prediction task are specified entirely in YAML configuration files, without writing Python, turning a collection of CSV or Parquet files into a RelArena-α task. To make RPI as accessible as possible, we provide extensive documentation, an agent skill, example specifications for all 21 entity-level tasks in RelBench v1, and a Kaggle-based example.

We acknowledge that designing an interface for specifying relational prediction problems remains an open research question, in part because the requirements of practitioners are not yet fully understood. For this initial version of RPI, we therefore adopt a deliberately expressive design aimed at researchers and early-adopting practitioners. The interface currently includes only limited safeguards against task mis-specification. This design reflects a broader tension between expressivity and usability: we expect that no single interface will be optimal for all settings and that different Pareto-optimal designs may serve different users, tasks, and levels of expertise. At the same time, the expressivity of RPI is intentionally constrained to entity-level forecasting tasks in its current version to ensure compatibility with RelArena-α. As a result, not every predictive task over a relational database can currently be represented within the framework. We are committed to working with the community to better understand the underlying trade-offs and to improve the accessibility, reliability, and expressivity of future versions of RPI. In particular, we have recently become aware of similar, independent efforts to establish an interface for specifying entity-level forecasting tasks and are planning to unify RPI's specification standard with this effort in the future.

3 Results

3.1 Experimental setup

General setup. We report the state of RelArena-α at release over the 21 entity-level RelBench v1 tasks at a single seed, once for model submissions (under the tuning regime of Appendix B) and once including system submissions in addition. We evaluate six model submissions: TabPFN-Rel (API), which runs the hosted model with text features, and TabPFN-Rel (OSS), which runs the open-source

model without them; RDBLearn [6] (v1)[1], GraphSAGE [27, 1] (also known as the "RDL" baseline), RelGT [16], and RelGNN [10].
Moreover, for completeness, we include a common trivial LightGBM baseline, which we included in the rank and Elo computation but omitted from the figures, with per-task scores in Tables 4 and 5. This trivial LightGBM baseline operates only on entity-level features [1], using the entity's own columns and the anchor timestamp but no aggregation over related tables. Likewise, we include two learning-free predictors, which predict the optimal constant for the task's primary metric (the positive-class rate for classification, the median for MAE), either globally (`constant-global`) or per entity (`constant-per-entity`).
We further evaluate RT-PluRel [20, 21] with a custom tuning regime as a system submission. The distinction between submission types is explained in Section 2.1. We fit and evaluate every method through the RelArena-α framework.

Known limitations. We acknowledge that standardizing tuning across methods is not fully solved in RelArena-α. In particular, while we made efforts to align runtimes across the different RelArena-α baselines, significant variance remains, which could advantage more expensive methods. We discuss this in detail in Appendix B, but note that TabPFN-Rel does not have a significantly higher runtime than the other model submissions.

Possible conflicts of interest. Releasing a benchmarking framework in addition to a model evaluated on said framework constitutes a conflict of interest. As stated in Section 2.1, RelArena-α currently uses the most widely used set of relational learning tasks [1], while applying benchmarking standards from the most widely used tabular benchmarking framework [5]. We explain and motivate the methodological choices underlying RelArena-α in Section 1 and Section 2.1; these choices try to reflect established practices rather than favoring any particular method. Moreover, by open-sourcing RelArena-α, TabPFN-Rel, and the custom RT-PluRel protocol, we make the benchmark design and our method implementations fully available for independent scrutiny and reproduction. This transparency allows questionable or potentially biased design choices, should any exist, to be readily identified, challenged, and evaluated under alternative benchmark configurations.

3.2 Elo rankings on RelArena-α

Figure 2 presents the Elo ratings over RelArena-α, computed using TabArena's `bencheval` [5]. Pairwise task outcomes across all method pairs are fit via maximum likelihood under the Bradley-Terry model [40, 28, 5]. Ratings are anchored to the global constant predictor at 1000 points (where a 400-point gap implies a $\approx 91\%$ win probability), with confidence intervals derived from bootstrapped battles. Appendix D elaborates on the full results, presenting also dataset-level raw scores.

Main Results. Figure 2 presents the main leaderboards of RelArena-α, separating model submissions from system submissions. Model submissions are tuned under RelArena-α's standardized, compute-matched tuning regime (Appendix B). System submissions respect RelArena-α's data states and evaluation regime but bring their own tuning protocol. RT-PluRel [20, 21] is the first such system, using a custom, sequential tuning regime. Within models with the same tuning regime, TabPFN-Rel ranks first. When including systems in the comparison, RT-PluRel with its custom tuning achieves the highest end-to-end predictive performance. RT-PluRel's strong performance shows the value of researching co-dependent model and tuning-regime improvements, while also prompting further investigation into how to transfer this insight to other models.

Tabular models are actually highly competitive. Contrary to prevailing (theoretical) beliefs in the relational learning community, tabular models such as the TabPFN-Rel variants and RDBLearn are competitive with relational deep learning baselines. This adds to the growing evidence [6, 7, 8, 9] that flattening the database into a table is a strong strategy that can compete with relational deep learning models. At the same time, relational models offer unique modeling advantages, raising the exciting question of how to combine the best of both worlds.

[1] Our RDBLearn evaluation excludes LimiX, one of the three tabular foundation models considered in the original RDBLearn sweep, because it is not available as an installable PyPI package and would require vendoring an additional codebase. This may modestly disadvantage RDBLearn relative to its full configuration while also reducing its tuning cost. RDBLearn v1.1 [9] was released after our baseline cut-off and will be evaluated in a future RelArena-α release.

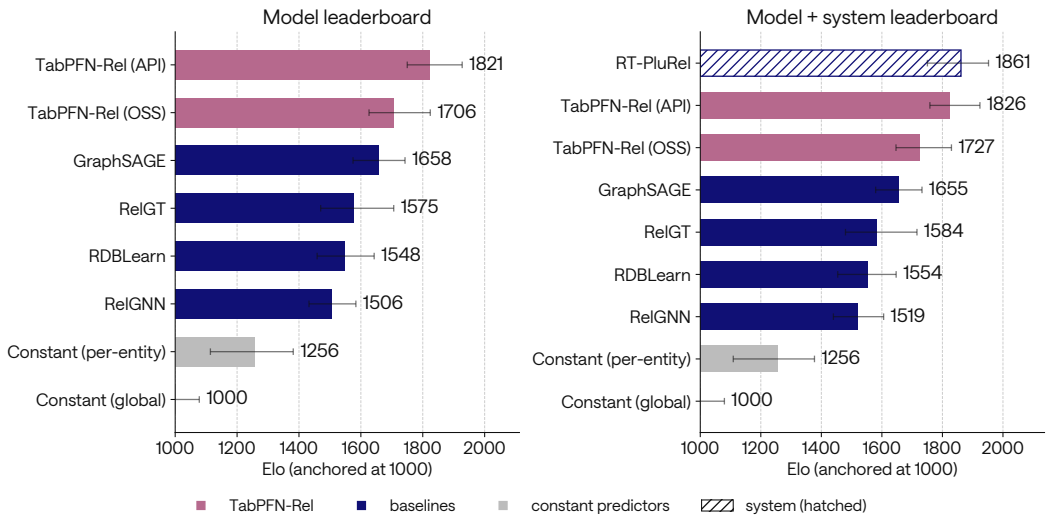


Figure 2: **Elo over the 21 RelArena-α tasks: the model leaderboard (left) and the combined model + system leaderboard (right).** Entries marked as systems (hatched) comply with RelArena-α's data states and evaluation regime but not with its standardized tuning regime (Appendix B). Each panel reports slightly different Elo for the same method, since Elo ratings are relative. TabPFN-Rel ranks first among models with the standardized tuning regime; the system submission RT-PluRel ranks first in end-to-end performance.

All methods are expensive to run. Table 2 in Appendix B presents the runtimes of RelArena-α. The single-seed leaderboard in Figure 2 required hundreds of hours of wall-clock time. For more expensive datasets, some methods are prohibitively slow, making them impractical for many real-world applications. Our method is not exempt: TabPFN-Rel requires a feature-synthesis pass over the database before it can be fit at all, and this CPU-bound preprocessing might dominate runtime depending on the dataset and hardware used. Other competitive methods are generally no cheaper to run: excluding preprocessing, which can be significantly more expensive for TabPFN-Rel, RT-PluRel's runtime is on average five times slower than TabPFN-Rel (API) and 30 times slower than TabPFN-Rel (OSS). We believe that reducing this cost is key for this field to mature to industry adoption.

Text columns are critical for some tasks. Running TabPFN-Rel without text features, as the OSS variant does, moves it from an Elo of 1821 to 1706, a drop larger than the gap between GraphSAGE and RelGT. The effect is not spread across the benchmark but dominated by two tasks: `rel-event/user-ignore` and `rel-avito/user-clicks`. Text is decisive where a task carries informative free-text columns and close to irrelevant elsewhere, as seen in Multimodal Tabular Learning benchmarks [41, 42, 43, 37].

Constant predictors are not trivially beaten. `constant-per-entity` predicts each entity's own optimal constant from its training history, using no features and no model, and is nevertheless better than RelGNN and RelGT on 4 tasks each. TabPFN-Rel and RT-PluRel are the only methods that exceed it on all 21. In addition, on the `rel-stack/post-votes` task, almost no method is significantly better than the constant predictor.

3.3 Tunability in relational learning

Figure 3 shows the improvement of tuning across all method–task pairs. We exclude RT-PluRel because it does not track the information required for this isolated investigation, as it is a system submission. We observe that RelGNN benefits significantly from tuning, while GraphSAGE shows smaller improvements. By contrast, for most methods in RelArena-α our tuning does not appear to lead to reliable improvements. Table 6 in the Appendix presents the breakdown per method.
The results have interesting implications for TabPFN-Rel and other flattening-based approaches.

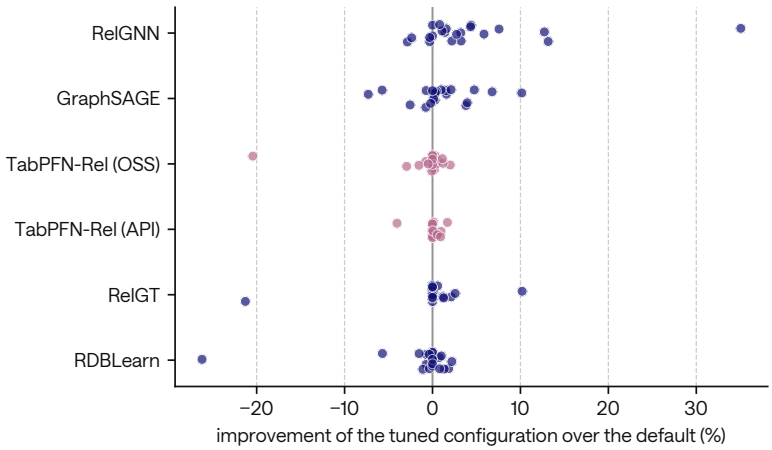


Figure 3: **Improvement of each method's tuned configuration over its own default.** One point per task and method, signed so that positive means tuning helped.

TabPFN-Rel searches only over aggregation depth (Appendix B), with the default corresponding to the shallowest depth considered. This default remains competitive with the deeper configurations selected by validation, suggesting that much of the predictive signal the method exploits is already accessible at depth two. Increasing the aggregation depth increases the number of join paths that must be materialized exponentially, leading to equivalently higher computational costs on larger databases. These results therefore suggest that simplifying the featurization procedure may be a promising avenue for improving the efficiency of TabPFN-Rel without substantially compromising predictive performance. A more systematic investigation of novel tuning strategies and their interaction with computational efficiency in relational learning remains open for future work.

4 Open Issues

While our contributions address several of the challenges identified in Section 1, important open issues remain. Some, such as expanding the set of baselines included in RelArena-α, can be addressed through continued framework development. Others raise broader research questions concerning the design, evaluation, and practical applicability of relational learning methods. In this section, we outline these challenges both to clarify the current limitations of RelArena-α and to identify promising directions for future work. Addressing them will require collaboration with the broader research community, and we hope that RelArena-α can provide a foundation for such efforts.

Running models is difficult. Running relational learning methods remains far more expensive and error-prone than running tabular ones. Most competitive methods require hours of CPU-bound preprocessing per dataset before model training or inference. Examples include deep feature synthesis for flattening-based methods like RDBLearn and TabPFN-Rel, or graph materialization and tokenization for GNN-based methods like RelGT and RelGNN. Including preprocessing in model training is possible but inefficient and impractical because CPU-based preprocessing and GPU-based model training have different hardware requirements. Additionally, runtime can vary by several orders of magnitude between methods. Both factors make fully standardizing tuning regimes across methods an open problem, which we discuss in Appendix B. Beyond benchmarking, broad adoption of relational learning methods requires making methods significantly cheaper and easier to run.

Data quality requires further investigation. With the release of RelArena-α, we have mainly focused on standardizing benchmarking on a pre-existing set of databases and tasks, namely the entity-level RelBench v1 tasks. While the selected tasks remain the most popular in the community, we believe it will be important to understand whether the underlying databases and tasks are of sufficient quality and representative of the tasks practitioners need to solve in their day-to-day work.

Missing baselines. As highlighted in Section 2.1, it is likely that competitive baselines are missing from RelArena-α for many possible reasons, such as differences in evaluation regimes, methods being released after our baseline cut-off date, or a lack of awareness on our part. The RelArena-α team will continuously work with authors to make the set of included baselines as representative as possible. In particular, we make it easy for authors to add their methods to RelArena-α by providing extensive documentation on the process as well as skill files for agentic coding tools. Stronger tabular baselines are likewise missing: we do not yet report baselines that lift traditional tabular learners to relational learning through automated feature engineering, as discussed in Section 1. Establishing how strong such baselines can be, and how much of the gap to specialized relational learning methods they close, is future work.

Disparate task types. RelArena-α currently limits itself to entity-level forecasting tasks [1]. While these tasks remain the most popular, the community has introduced many task types, including recommendation tasks [1], entity attribute prediction [2], relationship attribute prediction [2], foreign key prediction [2], and autocomplete tasks [4]. While many of these task types seem similar, they often have subtle differences that make them mutually incompatible, making cross-task method transfer difficult. We plan to extend RelArena-α to additional task types once we have a sufficient understanding of which task types represent real-world predictive tasks practitioners need to solve. Initial investigations suggest that not all task types fulfill this constraint equally.

Timestamp boundaries are not standardized. Following prior work, GNN-based baselines as well as the RT-PluRel system implemented in RelArena-α use rows at the entity's test timestamp as additional context, if test cut-off and entity timestamp align, while flattening-based approaches like TabPFN-Rel and RDBLearn do not. While this is currently permissible under RelArena-α's API, this might disadvantage the latter set of methods. At this time, it is unclear to the authors whether this additional context is permissible in general, only for certain tasks, or not at all, or how much it influences model predictions. We plan to align these boundaries between methods in future releases.

5 Outlook

The release of RelArena-α, TabPFN-Rel, and RPI marks only the first milestone of Prior Labs' long-term commitment to advancing the state of relational learning. For RelArena-α, we aim to work with the academic and open-source community to add baselines, further standardize tuning regimes, and address open questions about task types and data quality. The TabPFN-Rel harness will be further co-developed with our upcoming model releases, as well as based on community feedback. Additionally, we are exploring further ideas to apply Prior Labs' expertise in building foundation models for structured data to the relational domain. Lastly, we will iterate RPI based on community feedback to make relational learning approaches as accessible as possible to practitioners, closing the gap between academia and industry.

To conclude: towards non-dogmatic relational learning. With RelArena-α, TabPFN-Rel, and RPI, we are advancing open and reproducible relational learning. We are optimistic that our contributions can be an inflection point for the community, accelerating research progress and real-world impact. And, we strongly believe that this can only become reality when we, as the relational learning community, avoid dogmatic practices. A non-dogmatic relational learning community may be the key to lasting success by enabling more welcoming, rigorous, and collaborative research. Different sub-communities, such as graph, relational, or tabular researchers, would strongly benefit from working together and evaluating on the same, fair benchmarks.

References

[1] Joshua Robinson, Rishabh Ranjan, Weihua Hu, Kexin Huang, Jiaqi Han, Alejandro Dobles, Matthias Fey, Jan E. Lenssen, Yiwen Yuan, Zecheng Zhang, Xinwei He, and Jure Leskovec. RelBench: A Benchmark for Deep Learning on Relational Databases. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. doi: 10.48550/arXiv.2407.20060. URL http://arxiv.org/abs/2407.20060. arXiv:2407.20060 [cs.LG].

[2] Minjie Wang, Quan Gan, David Wipf, Zhenkun Cai, Ning Li, Jianheng Tang, Yanlin Zhang, Zizhao Zhang, Zunyao Mao, Yakun Song, Yanbo Wang, Jiahang Li, Han Zhang, Guang Yang, Xiao Qin, Chuan Lei, Muhan Zhang, Weinan Zhang, Christos Faloutsos, and Zheng Zhang. 4DBInfer: A 4D Benchmarking Toolbox for Graph-Centric Predictive Modeling on Relational DBs. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. doi: 10.48550/arXiv.2404.18209. URL http://arxiv.org/abs/2404.18209. arXiv:2404.18209 [cs.LG].

[3] Jakub Peleška and Gustav Šír. REDELEX: A Framework for Relational Deep Learning Exploration. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD 2025)*, volume 16014, pages 438–456, 2025. doi: 10.1007/978-3-032-05981-9_26. URL http://arxiv.org/abs/2506.22199. arXiv:2506.22199 [cs.LG].

[4] Justin Gu, Rishabh Ranjan, Charilaos Kanatsoulis, Haiming Tang, Martin Jurkovic, Valter Hudovernik, Mark Znidar, Pranshu Chaturvedi, Parth Shroff, Fengyu Li, and Jure Leskovec. RelBench v2: A Large-Scale Benchmark and Repository for Relational Data. In *3rd Workshop on Navigating and Addressing Data Problems for Foundation Models (DATA-FM) at ICLR 2026*, 2026. doi: 10.48550/arXiv.2602.12606. URL http://arxiv.org/abs/2602.12606. arXiv:2602.12606 [cs.LG].

[5] Nick Erickson, Lennart Purucker, Andrej Tschalzev, David Holzmüller, Prateek Mutalik Desai, David Salinas, and Frank Hutter. TabArena: A Living Benchmark for Machine Learning on Tabular Data. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025. doi: 10.48550/arXiv.2506.16791. URL http://arxiv.org/abs/2506.16791. arXiv:2506.16791 [cs.LG].

[6] Yanlin Zhang, Linjie Xu, Quan Gan, David Wipf, and Minjie Wang. RDBLearn: Simple In-Context Prediction Over Relational Databases, February 2026. URL http://arxiv.org/abs/2602.18495. arXiv:2602.18495 [cs.DB].

[7] Linjie Xu, Yanlin Zhang, Quan Gan, Minjie Wang, and David Wipf. No need to train your RDB foundation model. In *International Conference on Machine Learning (ICML)*, 2026. doi: 10.48550/arXiv.2602.13697. URL http://arxiv.org/abs/2602.13697.

[8] Xingyue Huang, Louis Tichelman, Jinwoo Kim, Krzysztof Olejniczak, and İsmail İlkan Ceylan. RelAgent: LLM Agents as Data Scientists for Relational Learning, May 2026. URL http://arxiv.org/abs/2605.07840. arXiv:2605.07840 [cs.LG].

[9] Linjie Xu and David Wipf. Parameter-Free Encoders Remain Viable for RDB Foundation Models, 2026. URL http://arxiv.org/abs/2607.05476. arXiv:2607.05476. ICML 2026 Workshop on Foundation Models for Structured Data.

[10] Tianlang Chen, Charilaos Kanatsoulis, and Jure Leskovec. RelGNN: Composite Message Passing for Relational Deep Learning. In *International Conference on Machine Learning (ICML)*, 2025. doi: 10.48550/arXiv.2502.06784. URL http://arxiv.org/abs/2502.06784. arXiv:2502.06784 [cs.LG].

[11] hydrogenhy. Request for releasing training code. GitHub issue #3 on snap-stanford/RelGNN, 2025. URL https://github.com/snap-stanford/RelGNN/issues/3.

[12] Valter Hudovernik, Federico López, Vid Kocijan, Akihiro Nitta, Jan Eric Lenssen, Jure Leskovec, and Matthias Fey. KumoRFM-2: Scaling Foundation Models for Relational Learning, April 2026. URL http://arxiv.org/abs/2604.12596. arXiv:2604.12596 [cs.LG].

[13] Kumo.AI. KumoRFM benchmark scripts. `benchmarks/` in `kumo-ai/kumo-rfm`, commit `e6c8ad5`, 2026. URL `https://github.com/kumo-ai/kumo-rfm`.

[14] Adrian Hayler. Additional details regarding the tuning regime. GitHub issue #4 on `snap-stanford/RelGNN`, 2026. URL `https://github.com/snap-stanford/RelGNN/issues/4`.

[15] Lennart Purucker. Source of RelBench benchmarking hparams (and recent changes)? Found on validation or test data? GitHub issue #74 on `kumo-ai/kumo-rfm`, 2026. URL `https://github.com/kumo-ai/kumo-rfm/issues/74`.

[16] Vijay Prakash Dwivedi, Sri Jaladi, Yangyi Shen, Federico López, Charilaos I. Kanatsoulis, Rishi Puri, Matthias Fey, and Jure Leskovec. Relational Graph Transformer. In *International Conference on Learning Representations (ICLR)*, 2026. doi: 10.48550/arXiv.2505.10960. URL `http://arxiv.org/abs/2505.10960`. arXiv:2505.10960 [cs.LG].

[17] Matthias Fey, Vid Kocijan, Federico Lopez, Jan Eric Lenssen, and Jure Leskovec. KumoRFM: A Foundation Model for In-Context Learning on Relational Data, May 2025. URL `http://web.archive.org/web/20260702201348/https://kumo.ai/research/kumo_relational_foundation_model.pdf`. Company whitepaper; archived copy, the original URL (`https://kumo.ai/research/kumo_relational_foundation_model.pdf`) is no longer online.

[18] Rishabh Ranjan, Vignesh Kothapalli, Harshvardhan Agarwal, Charilaos I. Kanatsoulis, Roshan Reddy Upendra, Tom Palczewski, Carlos Guestrin, and Jure Leskovec. Large-Scale Pretraining unlocks Few-Shot Prediction for Relational Data. In *2nd ICML Workshop on Foundation Models for Structured Data*, 2026. URL `https://openreview.net/forum?id=oQINTd9din`.

[19] Léo Grinsztajn, Klemens Flöge, Oscar Key, Felix Birkel, Philipp Jund, Brendan Roof, Mihir Manium, Shi Bin Hoo, Magnus Bühler, Anurag Garg, Dominik Safaric, Jake Robertson, Benjamin Jäger, Simone Alessi, Adrian Hayler, Vladyslav Moroshan, Lennart Purucker, Philipp Singer, Alan Arazi, Julien Siems, Jan Hendrik Metzen, Georg Grab, Nick Erickson, Siyuan Guo, Eliott Kalfon, Simon Bing, David Salinas, Clara Cornu, Lilly Charlotte Wehrhahn, Diana Kriuchkova, Kursat Kaya, Lydia Sidhoum, Marie Salmon, Jerry Chen, Madelon Hulsebos, Yann LeCun, Samuel Müller, Bernhard Schölkopf, Sauraj Gambhir, Noah Hollmann, and Frank Hutter. TabPFN-3: Technical Report, May 2026. URL `http://arxiv.org/abs/2605.13986`. arXiv:2605.13986 [cs.LG].

[20] Rishabh Ranjan, Valter Hudovernik, Mark Znidar, Charilaos Kanatsoulis, Roshan Upendra, Mahmoud Mohammadi, Joe Meyer, Tom Palczewski, Carlos Guestrin, and Jure Leskovec. Relational Transformer: Toward Zero-Shot Foundation Models for Relational Data. In *International Conference on Learning Representations (ICLR)*, 2026. URL `http://arxiv.org/abs/2510.06377`.

[21] Vignesh Kothapalli, Rishabh Ranjan, Valter Hudovernik, Vijay Prakash Dwivedi, Johannes Hoffart, Carlos Guestrin, and Jure Leskovec. PluRel: Synthetic Data unlocks Scaling Laws for Relational Foundation Models. In *International Conference on Machine Learning (ICML)*, 2026. URL `http://arxiv.org/abs/2602.04029`.

[22] The RelBench maintainers. Fix rel-event (user-ignore) and rel-stack (db), fix task names. `https://github.com/snap-stanford/relbench/pull/357`, 2026. GitHub pull request #357 on snap-stanford/relbench, merged 2026-02-01; regenerates the rel-event/user-ignore task table (temporal-leakage fix) and the rel-stack database; first contained in relbench 2.1.0.

[23] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. Pitfalls of Graph Neural Network evaluation, 2018. URL `http://arxiv.org/abs/1811.05868`. arXiv:1811.05868 [cs.LG].

[24] Arlind Kadra, Marius Lindauer, Frank Hutter, and Josif Grabocka. Well-tuned simple nets excel on tabular datasets. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL `https://proceedings.neurips.cc/paper_files/paper/2021/hash/c902b497eb972281fb5b4e206db38ee6-Abstract.html`.

[25] Bernd Bischl, Martin Binder, Michel Lang, Tobias Pielok, Jakob Richter, Stefan Coors, Janek Thomas, Theresa Ullmann, Marc Becker, Anne-Laure Boulesteix, Difan Deng, and Marius Lindauer. Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges. *WIREs Data Mining and Knowledge Discovery*, 13(2):e1484, 2023. doi: 10.1002/widm.1484.

[26] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. doi: 10.1609/aaai.v32i1.11694.

[27] Matthias Fey, Weihua Hu, Kexin Huang, Jan Eric Lenssen, Rishabh Ranjan, Joshua Robinson, Rex Ying, Jiaxuan You, and Jure Leskovec. Position: Relational Deep Learning: Graph Representation Learning on Relational Databases. In *International Conference on Machine Learning (ICML)*, 2024. doi: 10.48550/arXiv.2312.04615. URL http://arxiv.org/abs/2312.04615. arXiv:2312.04615 [cs.LG].

[28] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E Gonzalez, et al. Chatbot arena: An open platform for evaluating llms by human preference. In *International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 8359–8388. PMLR, 2024.

[29] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine learning research*, 7(Jan):1–30, 2006.

[30] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html.

[31] James Max Kanter and Kalyan Veeramachaneni. Deep feature synthesis: Towards automating data science endeavors. In *2015 IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, pages 1–10, 2015. doi: 10.1109/DSAA.2015.7344858.

[32] Hoang Thanh Lam, Johann-Michael Thiebaut, Mathieu Sinn, Bei Chen, Tiep Mai, and Oznur Alkan. One button machine for automating feature engineering in relational databases, 2017. URL https://arxiv.org/abs/1706.00327. arXiv:1706.00327 [cs.DB].

[33] getML. getML community edition: Feature learning for relational data and time series. https://github.com/getml/getml-community, 2022. GitHub repository; includes the FastProp propositionalization algorithm. Accessed 2026-08-02.

[34] Vid Kocijan, Jinu Sunil, Jan Eric Lenssen, Viman Deb, Xinwei Xe, Federico Reyes Gomez, Matthias Fey, and Jure Leskovec. Predictive Query Language: A Domain-Specific Language for Predictive Modeling on Relational Databases, February 2026. URL http://arxiv.org/abs/2602.09572. arXiv:2602.09572 [cs.DB]. Presented at TaDA@VLDB 2026.

[35] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Andreas Müller, Joel Nothman, Gilles Louppe, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL https://arxiv.org/abs/1201.0490.

[36] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-demos.6. URL https://aclanthology.org/2020.emnlp-demos.6/.

[37] Lennart Purucker, Andrej Tschalzev, Nick Erickson, Gioia Blayer, David Holzmüller, Alan Arazi, Alexander Pfefferle, Mustafa Tajjar, Gaël Varoquaux, and Frank Hutter. Beyond iid: How general are tabular foundation models, really? *arXiv preprint arXiv:2606.30410*, 2026.

[38] Kazi F Akhter, Bharath Ajendla, and Manar D Samad. A fair benchmarking of deep relational database learning models. *arXiv preprint arXiv:2607.03659*, 2026.

[39] Alteryx, Inc. Featuretools: An open source python library for automated feature engineering. `https://github.com/alteryx/featuretools`, 2017. GitHub repository, created September 2017; 7,666 stars and approx. 197,000 monthly PyPI downloads as of 2026-08-02.

[40] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3–4):324–345, 1952. doi: 10.1093/biomet/39.3-4.324.

[41] Gioia Blayer, Myung Jun Kim, Félix Lefebvre, Lennart Purucker, Alan Arazi, Eilam Shapira, Roi Reichart, Frank Hutter, Marine Le Morvan, David Holzmüller, et al. STRABLE: Benchmarking tabular machine learning with strings. *arXiv preprint arXiv:2605.12292*, 2026.

[42] Alan Arazi, Eilam Shapira, Shoham Grunblat, Mor Ventura, Elad Hoffer, Gioia Blayer, David Holzmüller, Lennart Purucker, Gaël Varoquaux, Frank Hutter, et al. MulTaBench: Benchmarking multimodal tabular learning with text and image. *arXiv preprint arXiv:2605.10616*, 2026.

[43] Alan Arazi, Eilam Shapira, and Roi Reichart. TabSTAR: A tabular foundation model for tabular data with text fields. *Advances in Neural Information Processing Systems*, 38:172108–172161, 2025.

[44] Jianmo Ni, Jiacheng Li, and Julian McAuley. Amazon Review Data (2018). Dataset page, University of California San Diego, 2018. URL `https://cseweb.ucsd.edu/~jmcauley/datasets/amazon_v2/`. No license stated; the page requests that Ni et al. (2019) be cited.

[45] Jianmo Ni, Jiacheng Li, and Julian McAuley. Justifying Recommendations using Distantly-Labeled Reviews and Fine-Grained Aspects. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 188–197, 2019. doi: 10.18653/v1/D19-1018.

[46] Kaggle. Avito Context Ad Clicks. Kaggle competition, 2015. URL `https://www.kaggle.com/competitions/avito-context-ad-clicks`. Competition rules require acceptance before download and are not publicly readable.

[47] Kaggle. Event Recommendation Engine Challenge. Kaggle competition, 2013. URL `https://www.kaggle.com/competitions/event-recommendation-engine-challenge`. Competition rules require acceptance before download and are not publicly readable.

[48] Ergast. Ergast Developer API: Terms & Conditions. Ergast Developer API, 2024. URL `http://web.archive.org/web/20240130210613/http://ergast.com/mrd/terms/`. Archived 30 January 2024. The API was deprecated during 2024 and shut down at the end of that year; `ergast.com` no longer serves the API or these terms.

[49] Kaggle. H&M Personalized Fashion Recommendations. Kaggle competition, 2022. URL `https://www.kaggle.com/competitions/h-and-m-personalized-fashion-recommendations`. Competition rules require acceptance before download and are not publicly readable.

[50] Stack Exchange. Stack Exchange Data Dump. The Internet Archive, 2023. URL `https://archive.org/download/stackexchange`. The `license.txt` accompanying the dump states that contributed content is "cc-wiki" licensed, linking to CC BY-SA 3.0, and sets out attribution requirements for republishing content.

[51] ClinicalTrials.gov. ClinicalTrials.gov Terms and Conditions. U.S. National Library of Medicine, 2023. URL `http://web.archive.org/web/20230612201546/https://clinicaltrials.gov/ct2/about-site/terms-conditions`. Archived 12 June 2023; the current page is rendered client-side and is not archivable as text.

A Authorship and Contributions

Adrian Hayler is listed first among the core contributors. The order of the remaining core contributors, Klemens Flöge and Alan Arazi, was randomized. Rishabh Ranjan contributed the RT-PluRel system submission. Aside from this, the external contributors Rishabh Ranjan and Jure Leskovec, as well as our scientific advisors listed below, contributed scientific advice only. They did not participate in the design or implementation of RelArena-α and contributed no intellectual property to it. Lennart Purucker, Frank Hutter, and Noah Hollmann led and supervised the project. The rest of the Prior Labs team is listed below, appearing in random order per group.

Research/Model Dev: Adrian Hayler, Klemens Flöge, Alan Arazi, Lennart Purucker, Oscar Key, Philipp Jund, Vahid Balazadeh, Felix Birkel, Mihir Manium, Léo Grinsztajn, Arthur Cahu, Siyuan Guo, Tobias Schroeder, Jonas Kübler, David Salinas, Jan Hendrik Metzen, Anurag Garg, Jake Robertson, Benjamin Jäger, Nick Erickson, Simon Bing.

Engineering/Platform: Dominik Safaric, Simone Alessi, Brendan Roof, Georg Grab.

Applied: Philipp Singer, Eliott Kalfon.

GTM: Clara Cornu, Vitor Monteiro, Diana Kriuchkova, Tuana Çelik, Lilly Wehrhahn.

Ops/People: Kürşat Kaya, Jerry Chen, Lydia Sidhoum, Rylee Grace, Marie Salmon, Tomás Pereda, Kristina Collins.

Scientific Advisors: Yann LeCun, Bernhard Schölkopf, Madelon Hulsebos.

Founders: Sauraj Gambhir, Frank Hutter, Noah Hollmann.

B Tuning Regime in RelArena-α

This appendix expands on the tuning regime summarized in Section 2.1. We describe how tuning is implemented in RelArena-α, the runtime policy that governs it, and the remaining challenges in making tuning fully comparable across methods. Such comparability requires standardization along two dimensions:

(i) *How* methods are tuned: which data and metric are used for model selection, and which search spaces are admissible.

(ii) *How much* methods are tuned: how much computation may be devoted to hyperparameter search.

We believe that RelArena-α largely addresses (i). By contrast, (ii) remains unresolved. Although the current runtime policy represents substantial progress, we believe that further iterations are needed for a fully satisfactory solution, likely requiring broader involvement of the academic community.

Tuning procedure. Each method registers a search space in a standardized format together with a default configuration. The search space is either sampled randomly, using the run seed, or specified as a small fixed grid whose configurations are evaluated in a predefined order. RelArena-α then performs tuning automatically. For each configuration, it fits the method on the inner split's training data and evaluates it on the corresponding validation data using the task's primary metric. The configuration with the best validation score is selected and refit on the outer split for final evaluation. The default configuration is refit under the same protocol, so each method reports both an untuned and a tuned result.

Methods additionally specify whether the final fit combines the training and validation data or retains the validation split for early stopping, following the protocol used in the corresponding publication. Search spaces may not be tailored to individual datasets, except through coarse tiers based on dataset size. At present, only RelGT uses such tiers, allowing us to reproduce the authors' implementation. We are planning to re-evaluate whether any adjustment of search spaces, based on dataset size, is necessary.

Challenges in standardizing budgets. Two properties of current relational learning methods make it difficult to standardize tuning compute.

First, runtimes on individual datasets can be prohibitively long. For example, under the authors' nine-configuration tuning regime, RelGT requires more than 40 hours on an RTX 6000 Pro GPU for `rel-avito/user-visits`, even when excluding CPU-based preprocessing that itself takes several hours. By comparison, the tuned LightGBM baseline completes the same task in under 30 seconds.

Second, many competitive methods rely on CPU-bound preprocessing, which can often take hours per dataset. Examples include deep feature synthesis for flattening-based methods like RDBLearn and TabPFN-Rel and graph materialization or tokenization for GNN-based methods like RelGNN and RelGT. We initially implemented all methods as end-to-end baselines whose preprocessing was executed directly within RelArena-α. We ultimately had to abandon this design because preprocessing is predominantly CPU-bound, whereas most GPU nodes provide comparatively weak CPUs. At the scale of our experiments, using GPU nodes for hundreds of hours of CPU preprocessing would have been difficult to justify economically, both for industry practitioners and ourselves.

These observations suggest that most current relational learning methods are not well suited to end-to-end execution on a single hardware configuration. RelArena-α therefore permits methods to compute preprocessing artifacts once and cache them on disk before a run. The corresponding cache-generation scripts must be public, allowing others to reconstruct the caches and enabling reviewers to inspect them for errors such as label leakage. Cache generation is currently done primarily via free-form scripts that are not directly linked to the RelArena-α framework. In particular, this means we currently do not track runtimes for cache generation, which also includes nuances such as how to amortize the runtime of preprocessing that is shared among the different tasks of one database. We regard this as a limitation of the current release and plan to address it in future versions.

Current runtime policy. We currently impose a maximum total runtime of 24 hours per task, including preprocessing, measured on the largest tasks. Moderately larger search spaces are permitted on smaller tasks when their cost remains reasonable. In practice, all released baseline models, except RelGT, run for less than 12 hours per task. Because these constraints are still approximate, we ask method developers to exercise reasonable judgment, and we encourage open discussion. System submissions must conform to the same runtime constraints, but sit outside the standardized tuning regime: RT-PluRel is not tuned through RelArena-α's trial budget at all — it runs a single configuration and selects its training step and context internally during each fit. Its recorded runtimes reach roughly 16 hours per task, within the 24-hour constraint; they are listed separately in Table 2. We permit such submissions in the current early version of RelArena-α, but may tighten the requirements on system submissions in the long term, depending on where the academic consensus settles.

Released configurations. Table 2 summarizes the trial budgets and runtimes of the released baselines. Trial budgets are specified per run rather than inferred automatically from the search spaces, and they reflect two different considerations. For methods with small fixed grids such as `rdblearn`, `tabpfn-rel` variants, or `relgt`, the budget is equal to the grid size, and the entire grid is evaluated with the default configuration first. For methods with sampled search spaces, the budget is chosen primarily according to computational cost. The inexpensive `lightgbm` baseline receives 30 trials, whereas the GPU-bound `graphsage` and `relgnn` baselines receive 4 and 10 trials, respectively. We note that the increased runtime of `tabpfn-rel-client` can be mostly attributed to calling TabPFN through the API rather than additional text processing. This task-independent overhead is reflected in the relatively moderate increase in maximum-over-median runtime.

RelGT is exceptional in two respects. First, it is the only RDL baseline whose tuning procedure is publicly available, with other methods either not disclosing their tuning regime or not using any tuning. We therefore tried to follow the authors' tuning regime as closely as possible in our initial implementation. Second, its grid depends on dataset size, which means that the effective number of trials on larger tasks may be smaller than the nominal budget. Once a more principled approach to standardizing tuning budgets is available, we intend to rerun all methods, including RelGT, under the revised regime.

Table 2: Per-task runtime for methods on RelArena-α at the time of release, aggregated over the 21 entity-level RelBench v1 tasks. $n_{\max}$ indicates the maximum number of hyperparameter configurations a method tunes over per dataset. Runtimes cover all tuning trials plus both refits (default and selected configuration), but do not include the CPU-bound preprocessing, which is performed to varying degrees by all competitive methods, including TabPFN-Rel and RDBLearn, which perform heavy CPU-bound feature engineering as part of the preprocessing. **The reported runtime numbers should therefore be seen as a lower bound on the actual runtime and are not directly representative of end-to-end method runtime.** The system submission `rt-plurel` is listed below the rule; it runs no tuning trials, so its runtime covers the fit and refit described in Appendix F. We provide further details in Appendix B.

Model	$n_{\max}$	Mean	Min	p25	p50	p75	Max
`constant-global`	0	0.1 s	0.0 s	–	0.0 s	–	0.3 s
`constant-per-entity`	0	0.2 s	0.0 s	–	0.1 s	–	1.3 s
`tabpfn-rel-client`	3	76 min	18 min	72 min	88 min	94 min	108 min
`tabpfn-rel-local`	3	12 min	0.6 min	0.8 min	4 min	6 min	85 min
`graphsage`	4	47 min	2 min	9 min	35 min	83 min	141 min
`rdblearn`	6	11 min	0.5 min	2 min	7 min	11 min	50 min
`relgt`	9	511 min	40 min	175 min	273 min	466 min	2442 min
`relgnn-es`	10	73 min	1 min	6 min	28 min	99 min	336 min
`lightgbm`	30	14 min	0.1 min	0.2 min	2 min	28 min	53 min
`rt-plurel`	–	351 min	109 min	129 min	231 min	516 min	979 min

Table 3: **Comparison of Elo ratings from authors' self-reported results and results obtained under RelArena-α's standardized evaluation framework.** Elo is computed on a joint board containing both sets of results, together with the constant baselines, and anchored at the global constant baseline. Because the two sets of results were obtained under different evaluation and tuning regimes, as discussed in Section 1, these Elo differences should be interpreted as measures of discrepancy rather than differences in method quality. We exclude RDBLearn [6] because the authors do not report results for 5 of the 21 tasks, and RT-PluRel because no published results correspond to its RelArena-α configuration: the RT and PluRel papers [20, 21] evaluate earlier versions of the code without per-task fine-tuning of the PluRel-pretrained model.

Method	Elo	CI+	CI−
RelGNN (self-reported)	1949	+114	−76
TabPFN-Rel (API)	1851	+82	−79
TabPFN-Rel (v3 model report)	1810	+116	−76
RelGT (self-reported)	1806	+94	−82
GraphSAGE (RelArena-α)	1714	+90	−94
GraphSAGE (self-reported)	1691	+73	−73
RelGT (RelArena-α)	1605	+127	−113
RelGNN (RelArena-α)	1503	+79	−72
LightGBM (RelArena-α)	1365	+104	−131
LightGBM (self-reported)	1324	+93	−106
Constant (per-entity)	1262	+124	−139
Constant (global)	1000	+88	−185

C Comparison against self-reported results

In Table 3, we compare results obtained through RelArena-α with those reported by the original authors. We exclude RDBLearn from this analysis because results are not reported for 5 of the 21 RelBench v1 tasks.

Methods with weaker self-reported performance, such as GraphSAGE, improve moderately under RelArena-α, likely in part because they benefit from the standardized tuning procedure. By contrast, RelGNN's Elo decreases by nearly 450 points relative to its self-reported results. We believe this discrepancy is largely driven by differences in tuning: RelArena-α's standardized tuning budget may

be insufficient to explore RelGNN's huge configuration space as thoroughly as the authors' original tuning procedure (see Section 1.1 and Appendix B). In addition, because we had to re-implement RelGNN, implementation differences may contribute to the observed gap, despite following the authors' recommendations [11].

RelGT's Elo decreases more moderately, by around 200 points. The source of this discrepancy is less clear, as our implementation closely follows the authors' released implementation and already incurs substantially higher runtime than any other method in RelArena-α (see Appendix B). TabPFN-Rel, in contrast, improves slightly relative to the version reported in the TabPFN-3 model report [19].

More generally, differences between self-reported and RelArena-α results can arise for many reasons, including differences in evaluation and tuning regimes, implementation details, and errors in our own re-implementations. Going forward, we aim to work with the original authors to ensure that each method is represented as faithfully and fairly as possible within the constraints of RelArena-α.

D Results tables per Task

Tables 4 and 5 report each method's test performance on all 21 RelArena-α tasks, using AUROC for classification and MAE in each regression task's native units; every entry is the test score of the configuration selected on validation data. These per-task measurements determine the pairwise outcomes used to compute the Elo ratings and bootstrapped confidence intervals in Figure 2.

Table 4: Test AUROC per task for every method run in RelArena-α (higher is better, best per task in bold). Columns, left to right: TPR = `tabpfn-rel-client`, TPR-OSS = `tabpfn-rel-local`, RT-P = `rt-plurel`, GSage = `graphsage`, RDBL = `rdblearn`, RGNN = `relgnn-es`, RGT = `relgt`, LGBM = `lightgbm`, Const-e = `constant-per-entity`, Const-g = `constant-global`.

Task	TPR	TPR-OSS	RT-P	GSage	RDBL	RGNN	RGT	LGBM	Const-e	Const-g
rel-amazon/item-churn	0.8280	0.8279	**0.8327**	0.8305	0.8195	0.7856	0.8238	0.6622	0.7289	0.5000
rel-amazon/user-churn	0.7086	0.7024	**0.7135**	0.7046	0.6844	0.6943	0.7019	0.5171	0.6342	0.5000
rel-avito/user-clicks	0.6752	0.6145	0.5834	0.6087	**0.6788**	0.6676	0.6444	0.5642	0.5041	0.5000
rel-avito/user-visits	0.6680	0.6688	**0.6709**	0.6658	0.6596	0.6487	0.6621	0.5293	0.6027	0.5000
rel-event/user-ignore	**0.8787**	0.7014	0.8476	0.7587	0.6644	0.8054	0.7815	0.7772	0.8399	0.5000
rel-event/user-repeat	0.7593	0.7693	**0.7914**	0.7846	0.7441	0.7546	0.7344	0.7483	0.7518	0.5000
rel-f1/driver-dnf	**0.7322**	0.7145	0.7315	0.7172	0.7146	0.7261	0.7117	0.7303	0.6993	0.5000
rel-f1/driver-top3	0.7714	0.7929	0.7589	0.7260	0.7801	0.7589	**0.8108**	0.7389	0.5565	0.5000
rel-hm/user-churn	0.7052	**0.7057**	0.7044	0.6985	0.6984	0.6820	0.6895	0.5901	0.6480	0.5000
rel-stack/user-badge	0.8804	0.8635	**0.8916**	0.8887	0.7711	0.6206	0.5743	0.5380	0.7890	0.5000
rel-stack/user-engagement	0.9060	0.9058	0.8968	0.9056	0.8587	0.9051	**0.9067**	0.8118	0.8267	0.5000
rel-trial/study-outcome	**0.7647**	0.7306	0.7235	0.6862	0.7212	0.6574	0.6685	0.7150	0.5000	0.5000

Table 5: Test MAE per task in the task's native units (lower is better, best per task in bold). Columns, left to right: TPR = `tabpfn-rel-client`, TPR-OSS = `tabpfn-rel-local`, RT-P = `rt-plurel`, GSage = `graphsage`, RDBL = `rdblearn`, RGNN = `relgnn-es`, RGT = `relgt`, LGBM = `lightgbm`, Const-e = `constant-per-entity`, Const-g = `constant-global`.

Task	TPR	TPR-OSS	RT-P	GSage	RDBL	RGNN	RGT	LGBM	Const-e	Const-g
rel-amazon/item-ltv	46.8	47.8	**43.0**	49.2	49.0	52.5	48.7	55.8	65.4	64.2
rel-amazon/user-ltv	14.4	14.4	**13.9**	14.4	14.6	14.6	14.4	16.8	17.4	16.8
rel-avito/ad-ctr	**0.0311**	0.0314	0.0348	0.0390	0.0341	0.0426	0.0365	0.0412	0.0412	0.0431
rel-event/user-attendance	0.244	**0.239**	0.241	0.245	0.242	0.244	0.261	0.263	0.269	0.264
rel-f1/driver-position	3.769	**3.762**	3.818	4.011	3.889	4.266	4.766	4.106	4.104	4.399
rel-hm/item-sales	0.0605	0.0614	**0.0403**	0.0552	0.0671	0.0565	0.0532	0.0753	0.0780	0.0761
rel-stack/post-votes	0.0679	0.0680	**0.0635**	0.0649	0.0677	0.0679	0.0679	0.0661	0.0694	0.0679
rel-trial/site-success	0.413	0.386	0.410	**0.325**	0.486	0.340	0.370	0.438	0.441	0.462
rel-trial/study-adverse	39.8	42.6	**32.7**	44.3	44.0	46.3	44.1	44.6	57.5	57.5

E Effect of Tuning per Method

Table 6 counts, for each method, the tasks on which the configuration selected on validation scores better, identically, or worse on test than that method's own default. A task is unchanged almost

exactly when validation kept the default, so the middle column doubles as a count of the tasks where the search found nothing worth taking.

Table 6: **Effect of tuning, per method.** Number of the 21 tasks on which the selected configuration beats, matches, or loses to that method's own default on test. The constant predictors have no search space and so cannot move.

Method	Better	Same	Worse
RelGNN	15	2	4
GraphSAGE	14	1	6
LightGBM	14	1	6
TabPFN-Rel (API)	9	7	5
RDBLearn	8	6	7
TabPFN-Rel (OSS)	8	5	8
RelGT	7	11	3
Constant (per-entity)	0	21	0
Constant (global)	0	21	0

F The RT-PluRel System Submission

RT-PluRel is the first system submission in RelArena-α: it registers an empty search space and runs a single configuration, performing all model selection internally during each fit. It builds on the pre-trained relational transformer [20, 21, 18], an 85M-parameter model (12 blocks, model dimension 512) that represents a relational database as a set of cell tokens connected by column, row, and foreign-key attention, with text cells embedded by a frozen sentence encoder (`all-MiniLM-L12-v2`). We use the publicly released checkpoints pre-trained under the PluRel synthetic-data protocol [21], one per task type (classification and regression).[2] For each prediction, the model receives a context assembled from the entity's relational neighborhood via breadth-first expansion and random-walk ranking; the context is bounded by the split's cut-off timestamp, so tuning and evaluation respect RelArena-α's data states (cf. Section 1.2 on the timestamp boundary).

Sequential tuning regime. Each fit proceeds in two stages. *Stage 1: fine-tuning with early stopping.* The pre-trained checkpoint is delta-fine-tuned on the task's training split—a zero-initialized additive delta on frozen pre-trained weights, so that weight decay regularizes toward the pre-trained model—for up to 50,000 steps (Muon optimizer, constant learning rate $5 \cdot 10^{-4}$, batch size 256, stochastic weight averaging). Training batches mix context configurations sampled from a grid over total context size $\{128, 256, 512, 1024\}$ (in cells), local context size, neighborhood width $\{16, 64, 256\}$, and a recency-preference flag. Every 100 steps, the averaged weights are evaluated on 1,024 validation rows under two endpoint context configurations; the best checkpoint under the task's primary metric is retained, and training stops after 10,000 steps without improvement. *Stage 2: context selection.* With the selected checkpoint frozen, 60 context configurations from the same grid are scored on up to 4,096 validation rows with four context-sampling seeds each, and the best configuration under the primary metric is selected.

Refit and prediction. Following RelArena-α's refit protocol, the model is then retrained from the pre-trained checkpoint on the union of training and validation data, scaling the selected step count by the ratio of dataset sizes, without any further model selection. Test predictions use the selected context configuration and ensemble eight context-sampling seeds, averaging raw outputs before the final sigmoid or denormalization. Because both stages select on validation data internally, RT-PluRel does not fit RelArena-α's standardized trial-budget accounting and is therefore reported as a system submission; see Appendix B for the runtime discussion.

[2] `https://huggingface.co/stanford-star/rt-plurel`

G Data Provenance

RelArena-α distributes no data. All seven RelBench v1 databases are obtained at runtime through the `relbench` package, and each remains subject to the terms of its original source. Table 7 lists the origin of each database, following the account of provenance given by RelBench [1]. Anyone using RelArena-α is bound by them directly and should consult the sources before relying on these databases for purposes of their own.

Table 7: Origin of the RelBench v1 databases evaluated in RelArena-α.

Database	Origin
`rel-amazon`	Amazon Review Data dump [44, 45]
`rel-avito`	Kaggle Avito Context Ad Clicks competition [46]
`rel-event`	Kaggle Event Recommendation Engine Challenge [47]
`rel-f1`	Ergast Developer API, retrieved February 2024 [48]
`rel-hm`	Kaggle H&M Personalized Fashion Recommendations competition [49]
`rel-stack`	Stack Exchange data dump via the Internet Archive, November 2023 [50]
`rel-trial`	ClinicalTrials.gov, January 2024 snapshot [51]

Attribution for `rel-stack`. The `rel-stack` database is built from content contributed by Stack Exchange users under CC BY-SA 3.0. The license distributed with the dump sets out its attribution requirements for republishing that content, which RelArena-α does not do. We nonetheless acknowledge the Stack Exchange communities and the individual contributors whose questions and answers make up this database, since their work is what the corresponding results measure.

Acknowledgment for `rel-event`. The Event Recommendation Engine Challenge data was made available for academic use with the explicit consent of its creators, and we join RelBench in thanking Allan Carroll for sharing it with the academic community [1]. That consent was given to RelBench, and we use the database only as RelBench distributes it.

Retrieval route. RelBench states that its users obtain the `rel-avito`, `rel-hm` and `rel-event` data from Kaggle themselves, and identifies accepting the competition terms as part of that step [1].

H Acknowledgements

We acknowledge the EuroHPC Joint Undertaking for awarding this project access to the EuroHPC supercomputer LUMI, hosted by CSC (Finland) and the LUMI consortium through a EuroHPC Regular Access call.

This work was supported by European Union's Horizon Europe research and innovation programme under grant agreement number 101214398 (ELLIOT).

Disclaimer: Funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Commission. Neither the European Union nor the European Commission can be held responsible for them.

